

powerbrmsINLA

A Comprehensive Guide to Bayesian Power Analysis

For Sport and Exercise Scientists

Version 1.2.0

CRAN Release — 2 June 2026

Table of Contents

1. Introduction
 - 1.1 What is powerbrmsINLA?
 - 1.2 Why Bayesian Power Analysis?
 - 1.3 Key Concepts
2. Installation and Setup
 - 2.1 Prerequisites
 - 2.2 Installing powerbrmsINLA
 - 2.3 Verifying Installation
3. Understanding Bayesian Power Analysis
 - 3.1 The Bayesian Workflow for Study Design
 - 3.2 Prior Distributions
 - 3.3 Decision Criteria Explained
 - 3.4 Comparison with Traditional Power Analysis
 - 3.5 Obtaining Traditional-Style Results
4. Core Functions
 - 4.1 `brms_inla_power()`: The Main Engine
 - 4.2 `brms_inla_power_sequential()`: Adaptive Stopping
 - 4.3 `brms_inla_power_two_stage()`: Adaptive Refinement
 - 4.4 `brms_inla_power_parallel()`: Multi-Core Computation
 - 4.5 `brms_inla_power_design()`: Unified Interface
5. Assurance and Decision Functions (New in 1.2.0)
 - 5.1 `compute_assurance()`
 - 5.2 `assurance_prior_weights()`
 - 5.3 `decide_sample_size()`
 - 5.4 `validate_inla_vs_brms()`
 - 5.5 `validate_sd_spec()`
6. Variance Uncertainty Integration (New in 1.2.0)
7. Prior Translation (New in 1.2.0)
8. Supported Statistical Models
9. Visualisation Functions
10. Quick-Start Example
11. Worked Examples for Sport and Exercise Science
12. Reporting Results
13. Computational Considerations
14. Approximate Analogues to Traditional Tools
15. Strengths and Limitations
16. Quick Reference
17. Glossary
18. References

1. Introduction

1.1 What is powerbrmsINLA?

powerbrmsINLA is an R package designed for Bayesian power analysis and sample size determination. It combines the intuitive model specification syntax of the brms package with the computational efficiency of INLA (Integrated Nested Laplace Approximation) to enable rapid simulation-based power calculations for a wide range of statistical models.

The package addresses a critical gap in the research planning toolkit: whereas traditional frequentist power analysis tools are readily available, Bayesian alternatives have historically been computationally prohibitive. By leveraging INLA's speed advantages over full Markov Chain Monte Carlo (MCMC) sampling, powerbrmsINLA makes Bayesian power analysis practical for routine use.

Version 1.2.0 introduces formal unconditional Bayesian assurance (O'Hagan & Stevens, 2001), automatic brms-to-INLA prior translation, variance uncertainty integration, and a comprehensive suite of visualisation functions.

1.2 Why Bayesian Power Analysis?

Traditional power analysis answers the question: "What sample size do I need to detect an effect with 80% probability, assuming a true effect of a given size?" Bayesian power analysis, often termed "assurance" or "probability of success," takes a more nuanced approach by:

- Incorporating prior uncertainty about the true effect size
- Allowing for multiple decision criteria beyond simple significance testing
- Providing probability statements about meeting specific inferential goals
- Enabling sensitivity analyses across different prior specifications

For sport and exercise scientists, this approach offers particular advantages. Effect sizes in our field are often uncertain, population heterogeneity is substantial, and the distinction between statistical significance and practical significance is paramount. Bayesian methods allow researchers to specify what magnitude of effect would be practically meaningful and to calculate the probability of obtaining conclusive evidence about such effects.

1.3 Key Concepts

1.3.1 Conditional Power vs Unconditional Assurance

Conditional Bayesian power is the probability that a prespecified decision criterion is met, given a fixed true effect size. This is what `brms_inla_power()` computes at each point on the effect grid.

Unconditional Bayesian assurance (O'Hagan & Stevens, 2001) averages conditional power over a *design prior* — a distribution representing genuine uncertainty about the true effect. This is computed by `compute_assurance()` in version 1.2.0. Assurance provides a more realistic assessment of study success probability because it does not condition on a single assumed effect size.

Terminology convention in this guide: We use "conditional power" or "conditional success probability" when referring to simulation results at a fixed effect value, and "assurance" exclusively for the unconditional quantity averaged over a design prior.

1.3.2 Decision Criteria

powerbrmsINLA supports multiple decision criteria for determining study "success":

- **Direction:** Posterior probability that the effect is in the expected direction (e.g., treatment improves outcome)
- **Threshold:** Posterior probability that the effect exceeds a meaningful threshold
- **ROPE (Region of Practical Equivalence):** Probability that the effect falls outside a region of negligible effects
- **Bayes Factor:** Evidence ratio comparing the alternative to null hypothesis
- **Precision:** Probability that the credible interval width is below a target

1.3.3 INLA for Speed

INLA provides deterministic approximations to posterior distributions and can be substantially faster than full MCMC for many latent Gaussian models, especially when repeated model fits are required (Rue et al., 2009). The speed advantage is model-dependent, and users should validate the approximation for non-standard models using `validate_inla_vs_brms()`.

2. Installation and Setup

2.1 Prerequisites

Before installing powerbrmsINLA, you need R (version 4.1.0 or later) and several dependencies.

2.1.1 Installing R

If you are new to R, download and install it from <https://cran.r-project.org/>. We also recommend RStudio (<https://posit.co/download/rstudio-desktop/>) as an integrated development environment.

2.1.2 Installing INLA

INLA is not on CRAN and must be installed separately. The following code installs it if it is not already available:

```
if (!requireNamespace("INLA", quietly = TRUE)) {
  install.packages(
    "INLA",
    repos = c(
      getOption("repos"),
      INLA = "https://inla.r-inla-download.org/R/stable"
    ),
    dep = TRUE
  )
}
```

2.2 Installing powerbrmsINLA

The package is available on CRAN:

```
# Install from CRAN
install.packages("powerbrmsINLA")

# Load the package
library(powerbrmsINLA)
```

2.3 Verifying Installation

Run the following code to verify versions and that all components are working:

```
library(powerbrmsINLA)
library(INLA)
library(brms)

# Check versions
packageVersion("powerbrmsINLA") # Should be >= 1.2.0
packageVersion("INLA")

# Quick test with a simple model
test_result <- brms_inla_power(
  formula      = y ~ treatment,
  effect_name  = "treatment",
  effect_grid  = 0.5,
  sample_sizes = 50,
  nsims        = 10,
  seed         = 123
)

print(test_result)
```

3. Understanding Bayesian Power Analysis

3.1 The Bayesian Workflow for Study Design

Bayesian sample size determination fits within a broader framework of principled study design:

1. Specify the research question and define what constitutes meeting a prespecified decision criterion
2. Identify the statistical model appropriate for the data structure
3. Elicit prior distributions reflecting existing knowledge about parameters
4. Define decision criteria (e.g., 95% posterior probability that effect exceeds threshold)
5. Simulate data across a range of sample sizes and effect magnitudes
6. Calculate conditional power at each grid point, then (optionally) unconditional assurance
7. Select the sample size that achieves acceptable assurance levels
8. Conduct sensitivity analyses to assess robustness of conclusions

3.2 Prior Distributions

Prior distributions encode existing knowledge or beliefs about model parameters before observing new data. In power analysis, priors serve two distinct roles:

3.2.1 Design Priors (Generating Priors)

These priors represent uncertainty about the true effect size that might be observed. They are used to weight conditional power estimates across the effect grid when computing unconditional assurance. For example, if previous studies suggest a treatment effect of approximately 0.5 standard deviations but with considerable uncertainty, you might use a Normal distribution centred at 0.5 with standard deviation 0.2. In version 1.2.0, these are formally specified via `assurance_prior_weights()` and used by `compute_assurance()`.

3.2.2 Analysis Priors

These priors are used when fitting the statistical model to each simulated dataset. They should reflect the prior knowledge that would actually be used in the final analysis. Weakly informative priors are often appropriate, providing regularisation without strongly influencing conclusions. In version 1.2.0, brms-style analysis priors are automatically translated to INLA controls via the built-in prior bridge.

Important: The design prior is used to generate or weight possible true effects for sample-size planning. The analysis prior is the prior used to fit each simulated dataset. These may be the same distribution, but they answer different questions and should be justified separately. Users can easily confuse a prior over plausible study effects with the prior they intend to use in the final model.

3.3 Decision Criteria Explained

3.3.1 Directional Probability

The simplest criterion asks: "What is the posterior probability that the effect is in the expected direction?" For a treatment expected to improve performance:

```
P(effect > 0 | data)
```

A study might be considered successful if this probability exceeds 0.95 or 0.975.

3.3.2 Threshold Probability

More stringent than directional, this criterion asks: "What is the posterior probability that the effect exceeds a practically meaningful threshold?"

```
P(effect > 0.2 | data) # Effect larger than 0.2 SDs
```

This directly addresses the distinction between statistical and practical significance.

3.3.3 ROPE (Region of Practical Equivalence)

The ROPE approach defines a region of effect sizes considered practically negligible. A study is successful if there is high probability the effect falls outside this region:

```
P(effect < -0.1 OR effect > 0.1 | data) # Outside ROPE of [-0.1, 0.1]
```

ROPE-based decisions can be framed either as evidence *against* practical equivalence (when most posterior mass lies outside the ROPE) or as evidence *for* practical equivalence (when most posterior mass lies inside the ROPE). Users should specify the intended decision rule before simulation.

3.3.4 Bayes Factors

The Bayes factor quantifies evidence for one hypothesis relative to another. A Bayes factor BF_{10} of 10 indicates the data are 10 times more likely under the alternative than null hypothesis. Conventional thresholds include:

- $BF_{10} > 3$: Moderate evidence
- $BF_{10} > 10$: Strong evidence
- $BF_{10} > 30$: Very strong evidence

These thresholds should be treated as conventions, not universal decision rules. Bayes factor results are sensitive to the prior on the tested parameter, particularly when using point-null comparisons.

powerbrmsINLA supports two Bayes factor methods: Savage-Dickey density ratio (`bf_method = "sd"`) and marginal likelihood (`bf_method = "marglik"`). The marginal likelihood method, new in 1.2.0, is more general but slower.

3.3.5 Precision-Based Criteria

Rather than focusing on hypothesis testing, precision-based criteria ask: "What is the probability that the credible interval will be narrow enough to be informative?" This aligns with estimation-focused research goals:

```
P(credible interval width < 0.3 | data) # 95% CrI narrower than 0.3 SDs
```

3.4 Comparison with Traditional Power Analysis

The following table summarises key differences between frequentist power analysis and Bayesian assurance:

Aspect	Frequentist Power	Bayesian Assurance
Effect size	Fixed, assumed known	Uncertain, described by design prior
Success criterion	$p < 0.05$ (typically)	Multiple options (posterior probability, BF, precision)
Prior knowledge	Not formally incorporated	Formally specified via priors
Interpretation	Long-run rejection rate	Probability of meeting inferential goal
Complex models	Limited support	Broad GLM/GLMM support

3.5 Obtaining Traditional-Style Results

For researchers familiar with G*Power or JASP who wish to obtain comparable results, powerbrmsINLA can approximate traditional power by:

1. Using a single effect size value (no prior uncertainty)
2. Setting the direction threshold to approximate a one-sided test
3. Using weakly informative analysis priors

Example comparing with a two-sample t-test:

```
# Traditional approach: fixed effect size of 0.5 (Cohen's d)
result <- brms_inla_power(
  formula = y ~ group,
  effect_name = "group",
  effect_grid = 0.5,          # Single effect size
  sample_sizes = c(30, 50, 70, 100),
  nsims = 500,
  prob_threshold = 0.975,
  seed = 123
)
```

Note on frequentist approximation: With weak analysis priors and approximately normal posteriors, requiring $P(\text{effect} > 0 \mid \text{data}) > 0.975$ approximates rejecting a two-sided frequentist test at $\alpha = 0.05$ when the expected effect direction is positive. This is an approximation, not an exact frequentist equivalence. It may not hold for asymmetric posteriors, non-Gaussian models, informative priors, or small samples.

4. Core Functions

4.1 brms_inla_power(): The Main Engine

This is the primary function for fixed sample size power analysis. It simulates datasets across a grid of sample sizes and effect values, fits INLA models to each, and summarises the proportion meeting various decision criteria. The output is **conditional** on each fixed effect-grid value. To obtain unconditional assurance, pass the result to `compute_assurance()`.

Version 1.2.0 adds multi-effect grids, brms-to-INLA prior translation, marginal-likelihood Bayes factors, distributional variance specifications, and automatic INLA thread detection.

4.1.1 Basic Syntax

```
result <- brms_inla_power(
  formula,                # Model formula (brms syntax)
  family = gaussian(),    # Response distribution
  effect_name,            # Name(s) of effect(s) to assess
  effect_grid,            # Values of effect(s) to simulate
  sample_sizes,           # Sample sizes to evaluate
  nsims = 200,            # Simulations per cell
  priors = NULL,          # brms prior specification
  prob_threshold = 0.95,   # Posterior probability threshold
  effect_threshold = 0,    # Effect size threshold
  rope_bounds = NULL,     # ROPE boundaries (length 2)
  credible_level = 0.95,  # Credible interval level
  compute_bayes_factor = FALSE,
  bf_method = "sd",       # "sd" (Savage-Dickey) or "marglik"
  bf_cutoff = 10,         # BF threshold for hit rate
  error_sd = 1,           # Residual SD (scalar or dist. list)
  group_sd = 0.5,         # Random effect SD (scalar or dist. list)
  inla_num_threads = NULL, # Auto-detected if NULL
  seed = 123,
  progress = "auto"
)
```

Scale of effect: `effect_grid` values must be specified on the scale of the model coefficient. For Gaussian models with `error_sd = 1`, these may correspond approximately to standardised mean differences. For binomial models they are log-odds effects; for Poisson/negative binomial models they are log-rate effects; and for survival models they depend on the family/link specification.

4.1.2 Return Value

The function returns a list of class "brms_inla_power" with these components:

- **results:** A data frame with per-simulation results including posterior means, SDs, probabilities, and credible interval widths
- **summary:** Aggregated conditional power metrics by sample size and effect grid
- **diagnostics:** INLA fitting diagnostics (proportion of warnings, failed fits)
- **settings:** A record of all analysis parameters, including prior_translation audit trail

4.1.3 Interpreting the Summary

The summary output contains several key columns. All values are **conditional** on the simulated effect value:

Column	Description
power_direction	Proportion of simulations where $P(\text{effect} > 0 \mid \text{data})$ exceeds prob_threshold, at the given effect value
power_threshold	Proportion where $P(\text{effect} > \text{effect_threshold} \mid \text{data})$ exceeds prob_threshold
power_rope	Proportion where $P(\text{effect outside ROPE} \mid \text{data})$ exceeds prob_threshold
avg_ci_width	Mean credible interval width across simulations
bf_hit_10	Proportion of simulations achieving $\text{BF}_{10} \geq 10$

4.1.4 Worked Example: Training Intervention

Consider a study examining whether a 12-week strength training intervention improves vertical jump performance. Based on prior research, we expect effects between 0.3 and 0.7 standard deviations.

```
library(powerbrmsINLA)
library(brms)

# Define analysis priors
priors <- c(
  prior(normal(0, 1), class = "Intercept"),
  prior(normal(0, 0.5), class = "b", coef = "training")
)

# Run conditional power simulations
vj_power <- brms_inla_power(
  formula = jump_height ~ training,
  family = gaussian(),
  effect_name = "training",
  effect_grid = c(0.3, 0.4, 0.5, 0.6, 0.7),
  sample_sizes = c(20, 30, 40, 50, 60, 80, 100),
  nsims = 500,
  priors = priors,
  prob_threshold = 0.95,
  effect_threshold = 0.2, # Minimum practically meaningful effect
  rope_bounds = c(-0.2, 0.2),
  compute_bayes_factor = TRUE,
  bf_method = "sd",
  seed = 42
)

# View conditional power summary
print(vj_power)

# Compute unconditional assurance
assurance <- compute_assurance(
  vj_power,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.15),
  metric = "threshold"
)
print(assurance)
```

4.2 brms_inla_power_sequential(): Adaptive Stopping

The sequential engine simulates data in batches, stopping early when conditional power can be estimated with sufficient precision. This reduces computation time by avoiding unnecessary simulations when conditional power is clearly high or low.

```
seq_result <- brms_inla_power_sequential(
  formula = y ~ treatment,
  effect_name = "treatment",
```

```

effect_grid = c(0.3, 0.5, 0.7),
sample_sizes = c(30, 50, 100),
metric = "direction",      # Which criterion to monitor
target = 0.80,             # Target conditional power
batch_size = 20,           # Simulations per batch
min_sims = 40,             # Minimum before stopping allowed
max_sims = 600,            # Maximum simulations per cell
margin = 0.02,             # Stopping margin around target
ci_conf = 0.95             # Confidence for stopping rule
)

```

4.3 brms_inla_power_two_stage(): Adaptive Refinement

This function implements a two-stage adaptive design: Stage 1 uses a coarse grid to identify promising regions; Stage 2 refines the grid around cells near the target.

```

effect_grid <- expand.grid(
  training = c(0.3, 0.5, 0.7),
  age_effect = c(-0.1, 0, 0.1)
)

two_stage <- brms_inla_power_two_stage(
  formula = performance ~ training + age,
  effect_name = c("training", "age"),
  effect_grid = effect_grid,
  n_range = c(30, 150),
  stage1_k_n = 6,
  stage1_nsims = 100,
  stage2_nsims = 400,
  refine_metric = "direction",
  refine_target = 0.80,
  band = 0.06,
  expand = 1
)

```

4.4 brms_inla_power_parallel(): Multi-Core Computation

For computationally intensive analyses, the parallel wrapper distributes simulations across multiple CPU cores.

```

parallel_result <- brms_inla_power_parallel(
  design = "fixed",
  formula = y ~ treatment + (1|subject),
  family = gaussian(),
  effect_name = "treatment",
  effect_grid = seq(0.2, 0.8, by = 0.1),
  sample_sizes = c(30, 50, 100, 150),
  nsims = 500,
  n_cores = 4,
  seed = 123
)

```

4.5 brms_inla_power_design(): Unified Interface

This wrapper function provides a unified interface to all three engines (set `design = "fixed"`, `"sequential"`, or `"two_stage"`).

5. Assurance and Decision Functions (New in 1.2.0)

5.1 compute_assurance(): Unconditional Bayesian Assurance

This function computes unconditional Bayesian assurance — the probability of a successful trial outcome averaged over prior uncertainty about the true effect size — from the output of `brms_inla_power()` or related engines. The method follows O'Hagan & Stevens (2001).

Formally, assurance at sample size n is:

$$A(n) = \sum_j w_j * \text{Power}(n, \text{delta}_j)$$

where w_j are normalised design-prior weights over the discrete effect grid.

5.1.1 Syntax

```
out <- compute_assurance(
  power_result,          # Output from brms_inla_power()
  prior_weights,         # Named vector or distribution list
  metric = "direction",  # "direction", "threshold", "rope", or "bf"
  weight_tol = 0.01
)
```

5.1.2 Prior Weight Formats

Two formats are accepted for `prior_weights`:

- **Named numeric vector:** Names must match effect-grid values. Example: `c("0.2" = 1/3, "0.5" = 1/3, "0.8" = 1/3)`
- **Distribution list:** `list(dist = "normal", mean = 0.5, sd = 0.2)`. Supported distributions: normal, uniform, beta.

5.2 assurance_prior_weights(): Design Prior Weights

A convenience wrapper that evaluates a parametric prior distribution at each value in the effect grid, normalises the densities to sum to 1, and returns a named numeric vector compatible with `compute_assurance()`.

```
effects <- c(0.1, 0.3, 0.5, 0.7, 0.9)

# Normal prior centred on 0.5
w_norm <- assurance_prior_weights(effects, dist = "normal",
                                  mean = 0.5, sd = 0.2)

# Uniform prior (equal weight)
w_unif <- assurance_prior_weights(effects, dist = "uniform")

# Beta prior
w_beta <- assurance_prior_weights(effects, dist = "beta",
                                   shape1 = 2, shape2 = 5)
```

5.3 decide_sample_size(): Automated Recommendations

This helper identifies the minimum sample size meeting specified targets. It operates in two modes:

- **Assurance mode** (recommended): Supply `prior_weights`. Returns the smallest n where unconditional assurance reaches each metric target.

- **Conditional mode** (backward compatible): When `prior_weights = NULL`, selects the smallest `n` at which per-cell conditional power meets the target for each effect-size value.

```
# Assurance mode
w <- assurance_prior_weights(c(0.2, 0.5, 0.8), dist = "normal",
                             mean = 0.5, sd = 0.2)

rec <- decide_sample_size(
  power_result,
  direction = 0.80,      # 80% assurance for direction
  threshold = 0.75,      # 75% assurance for threshold
  prior_weights = w
)
print(rec)
```

5.4 `validate_inla_vs_brms()`: Spot-Checking Posteriors

Generates a small number of synthetic datasets, fits each with both INLA and brms/Stan, and compares posterior estimates. This is a qualitative spot-check to verify that the INLA approximation is adequate for your specific model.

```
validation <- validate_inla_vs_brms(
  formula = y ~ treatment,
  family = gaussian(),
  effect_name = "treatment",
  effect_value = 0.5,
  sample_size = 100,
  n_check = 5,
  brms_iter = 2000,
  brms_chains = 4,
  seed = 42
)

print(validation$comparisons)
cat("Max absolute difference:", validation$max_abs_diff, "\n")
cat("Flag:", validation$flag, "\n")
```

5.5 `validate_sd_spec()`: Variance Specification Validation

Checks whether an `error_sd` or `group_sd` specification is valid. Called automatically by `brms_inla_power()` but can be used interactively.

```
# Scalar (always valid if positive)
validate_sd_spec(1.0)

# Distributional (validated against supported formats)
validate_sd_spec(list(dist = "halfnormal", sd = 0.3, location = 1.0))
validate_sd_spec(list(dist = "lognormal", meanlog = 0, sdlog = 0.5))
validate_sd_spec(list(dist = "uniform", min = 0.5, max = 1.5))
```

6. Variance Uncertainty Integration (New in 1.2.0)

When the residual variance itself is uncertain (e.g., estimated from a small pilot study), it is important to integrate over that uncertainty rather than conditioning on a single point estimate. Version 1.2.0 allows `error_sd` and `group_sd` to accept distributional specifications. At each simulation iteration, a fresh value is drawn from the specified distribution and used by the automatic data generator.

Supported Distributions

Format	Description
<code>list(dist = "halfnormal", sd = X, location = Y)</code>	Draws $ N(\text{location}, \text{sd}) $; location defaults to 0
<code>list(dist = "lognormal", meanlog = X, sdlog = Y)</code>	Draws from a log-normal distribution
<code>list(dist = "uniform", min = X, max = Y)</code>	Draws from $\text{Uniform}(\text{min}, \text{max})$; requires $\text{min} \geq 0$

Example

```
# Integrate over uncertainty in residual SD
result <- brms_inla_power(
  formula      = y ~ treatment,
  effect_name  = "treatment",
  effect_grid  = 0.5,
  sample_sizes = c(50, 100, 200),
  nsims        = 500,
  error_sd     = list(dist = "halfnormal", sd = 0.3, location = 1.0),
  group_sd     = list(dist = "uniform", min = 0.3, max = 0.8),
  seed        = 42
)

# The results data frame records sampled_error_sd and sampled_group_sd
# at each iteration. The summary reports their mean and SD.
```

7. Prior Translation (New in 1.2.0)

Version 1.2.0 includes an automatic translation layer that converts brms-style prior specifications to their INLA equivalents.

Supported Prior Classes

brms Prior	INLA Translation
<code>prior(normal(mu, sigma), class = "Intercept")</code>	<code>control.fixed: mean.intercept, prec.intercept</code>
<code>prior(normal(mu, sigma), class = "b", coef = "x")</code>	<code>control.fixed: mean, prec for named coefficient</code>
<code>prior(student_t(df, mu, sigma), class = "b")</code>	Approximated as <code>Normal(mu, sigma)</code> in INLA
<code>prior(exponential(rate), class = "sigma")</code>	Translated to <code>pc.prec</code> prior for observation precision
<code>prior(exponential(rate), class = "sd", group = "g")</code>	Translated to <code>pc.prec</code> prior for group-level precision

Unsupported prior classes are reported in the audit trail (`settings$prior_translation`) rather than silently ignored. Direct `family_control` settings always take precedence.

8. Supported Statistical Models

8.1 Response Distributions

Family	Use Case	Sport/Exercise Example
gaussian()	Continuous outcomes	Jump height, VO2max
binomial()	Binary/proportion data	Injury incidence, success rate
poisson()	Count data	Goals scored, training sessions
negbinomial()	Overdispersed counts	Injury counts with clustering
beta()	Proportions (0,1)	Percentage of training load
student()	Heavy-tailed continuous	Reaction times with outliers
lognormal()	Positive skewed data	Race times, force values
betabinomial()	Overdispersed proportions	Pass completion rates
weibull()	Survival/time-to-event	Time to return from injury
exponential()	Time between events	Inter-injury intervals

8.2 Random Effects

The package supports random effects using brms syntax, which is translated to INLA's $f()$ notation:

```
# Random intercepts for subjects
formula = y ~ treatment + (1 | subject)

# Random intercepts and slopes
formula = y ~ treatment + time + (1 + time | subject)

# Crossed random effects
formula = y ~ treatment + (1 | subject) + (1 | trial)
```

Note: Currently, only one random slope per random effect term is fully supported. Complex random effect structures may require additional configuration.

8.3 Multi-Effect Analyses

```
effect_grid <- expand.grid(
  treatment = c(0.3, 0.5, 0.7),
  time_effect = c(0.1, 0.2, 0.3)
)

result <- brms_inla_power(
  formula = y ~ treatment + time,
  effect_name = c("treatment", "time"),
  effect_grid = effect_grid,
  sample_sizes = c(50, 100),
  nsims = 300
)
```


9. Visualisation Functions

powerbrmsINLA provides a comprehensive suite of plotting functions. All functions return ggplot2 objects that can be further customised.

9.1 Conditional Power Plots

9.1.1 plot_power_contour()

Creates filled contour plots showing **conditional** Bayesian success probability for each fixed effect size and sample size. To obtain unconditional assurance, first use `compute_assurance()` with a design prior.

```
plot_power_contour(
  power_results,
  power_metric = "direction",
  x_effect = "treatment",
  y_effect = "n",
  power_threshold = 0.80,
  show_threshold_line = TRUE
)
```

9.1.2 plot_power_heatmap()

Similar to contour plots but using discrete tiles, useful when effect values are categorical or sparse. Shows conditional power at each grid cell.

```
plot_power_heatmap(
  power_results,
  power_metric = "threshold",
  x_effect = "training",
  y_effect = "n",
  facet_by = "age_group"
)
```

9.2 Unconditional Assurance Plots (New in 1.2.0)

9.2.1 plot_assurance_curve()

Plots unconditional assurance curves (from `compute_assurance()`) against sample size. Accepts multiple assurance objects for comparison.

```
a1 <- compute_assurance(pr, list(dist = "normal", mean = 0.5, sd = 0.15))
a2 <- compute_assurance(pr, list(dist = "normal", mean = 0.5, sd = 0.30))

plot_assurance_curve(list(a1, a2), labels = c("Informative", "Diffuse"))
```

9.2.2 plot_power_assurance_overlay()

Overlays the unconditional assurance curve on top of the per-effect conditional power curves.

```
assurance <- compute_assurance(pr, list(dist = "normal", mean = 0.5, sd = 0.15))
plot_power_assurance_overlay(pr, assurance)
```

9.2.3 plot_design_prior()

Visualises the design prior density over the effect grid.

```
plot_design_prior(
  prior_spec = list(dist = "normal", mean = 0.5, sd = 0.15),
  effect_grid = c(0.2, 0.5, 0.8)
```

)

9.3 Bayes Factor Visualisations

All BF plots show conditional results at each effect-grid value.

- **plot_bf_assurance_curve()** / **plot_bf_assurance_curve_smooth()**: Proportion of simulations achieving a BF threshold, with Wilson confidence intervals.
- **plot_bf_expected_evidence()**: Mean log10 BF across sample sizes and effect values.
- **plot_bf_heatmap()**: Two-dimensional heatmap of expected evidence strength.

9.4 Decision Criterion Plots

- **plot_decision_assurance_curve()**: Conditional success probability across effect sizes for different sample sizes.
- **plot_decision_threshold_contour()**: How conditional power changes across different decision thresholds (p^*).

9.5 Precision Plots

- **plot_precision_assurance_curve()**: Probability of achieving a target credible interval width.
- **plot_precision_fan_chart()**: Precision with confidence bands across sample sizes.

9.6 Other Plots

- **plot_assurance_with_robustness()**: Compares conditional power across multiple prior scenarios.
- **plot_interaction_surface()**: Visualises a metric as a function of two effect grid variables.
- **add_decision_overlay()**: Adds recommended sample size information to existing plots.

10. Quick-Start Example

This minimal example demonstrates the complete workflow from conditional power simulation to unconditional assurance, before the larger worked examples that follow.

Step 1: Conditional power simulation

```
library(powerbrmsINLA)

res <- brms_inla_power(
  formula      = y ~ treatment,
  family       = gaussian(),
  effect_name  = "treatment",
  effect_grid  = c(0.2, 0.5),
  sample_sizes = c(30, 60),
  nsims       = 50,
  seed        = 123
)

# Conditional power at each (effect, n) combination
res$summary
```

Step 2: From conditional power to unconditional assurance

```
# Average over a design prior on the effect size
assurance <- compute_assurance(
  res,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.2),
  metric = "direction"
)
print(assurance)
```

Step 3: Sample size recommendation

```
rec <- decide_sample_size(
  res,
  direction = 0.80,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.2)
)
print(rec)
```

11. Worked Examples for Sport and Exercise Science

11.1 Example 1: Randomised Controlled Trial

Research question: Does a novel periodisation strategy improve maximal strength compared to traditional linear periodisation?

Study Design

- Two-group parallel design (treatment vs control)
- Continuous outcome: 1RM squat (standardised)
- Expected effect: 0.4-0.6 SDs based on meta-analysis
- Goal: 80% probability of concluding effect exceeds 0.2 SDs

Analysis Code

```
library(powerbrmsINLA)
library(brms)

priors <- c(
  prior(normal(0, 1), class = "Intercept"),
  prior(normal(0.5, 0.3), class = "b", coef = "group")
)

rct_power <- brms_inla_power(
  formula = strength ~ group,
  family = gaussian(),
  effect_name = "group",
  effect_grid = seq(0.3, 0.7, by = 0.1),
  sample_sizes = seq(20, 100, by = 10),
  nsims = 500,
  priors = priors,
  prob_threshold = 0.95,
  effect_threshold = 0.2,
  rope_bounds = c(-0.2, 0.2),
  compute_bayes_factor = TRUE,
  seed = 123
)

# Conditional power contour
plot_power_contour(rct_power, power_metric = "threshold",
  power_threshold = 0.80)

# Unconditional assurance
assurance <- compute_assurance(
  rct_power,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.1),
  metric = "threshold"
)
print(assurance)

# Sample size recommendation
decide_sample_size(rct_power, threshold = 0.80,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.1))
```

11.2 Example 2: Repeated Measures Design

Research question: How does fatigue accumulate over the course of a training week in elite athletes?

```
repeated_power <- brms_inla_power(
  formula = fatigue ~ time + (1 | athlete),
  family = gaussian(),
  effect_name = "time",
  effect_grid = c(0.1, 0.15, 0.2, 0.25),
  sample_sizes = c(15, 20, 25, 30, 40),
  nsims = 400,
  error_sd = 1,
  group_sd = 0.5,
  obs_per_group = 5,
  prob_threshold = 0.95,
  seed = 456
)
```

11.3 Example 3: Injury Incidence Study

Research question: Does a preventive exercise programme reduce injury incidence in amateur footballers?

Note that effect_grid values are on the log-odds scale for binomial models:

```
injury_power <- brms_inla_power(
  formula = injury ~ intervention + (1 | team),
  family = binomial(),
  effect_name = "intervention",
  effect_grid = log(c(0.5, 0.6, 0.7)), # Log-odds scale
  sample_sizes = c(100, 150, 200, 300),
  nsims = 500,
  group_sd = 0.3,
  obs_per_group = 20,
  prob_threshold = 0.95,
  compute_bayes_factor = TRUE,
  seed = 789
)
```

11.4 Example 4: Precision-Focused Study

Research question: What sample size provides a sufficiently precise estimate of the relationship between training load and performance?

```
precision_power <- brms_inla_power(
  formula = performance ~ training_load,
  family = gaussian(),
  effect_name = "training_load",
  effect_grid = c(0.3, 0.4, 0.5),
  sample_sizes = seq(30, 150, by = 20),
  nsims = 500,
  precision_target = 0.25, # Target 95% credible interval width
  seed = 101
)
```

```
plot_precision_fan_chart(precision_power, ci_width_target = 0.25)
```

11.5 Example 5: Assurance with Design Prior

Research question: What sample size gives 80% unconditional assurance of detecting a treatment effect?

```
# Step 1: Conditional power simulations
power_result <- brms_inla_power(
```

```

    formula = y ~ treatment,
    effect_name = "treatment",
    effect_grid = seq(0.2, 0.8, by = 0.1),
    sample_sizes = seq(20, 120, by = 10),
    nsims = 500, seed = 42
  )

# Step 2: Compute unconditional assurance
assurance_info <- compute_assurance(
  power_result,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.15)
)

# Step 3: Sample size recommendation
rec <- decide_sample_size(
  power_result, direction = 0.80,
  prior_weights = list(dist = "normal", mean = 0.5, sd = 0.15)
)

# Step 4: Visualise
plot_assurance_curve(list(assurance_info))
plot_power_assurance_overlay(power_result, assurance_info)
plot_design_prior(
  list(dist = "normal", mean = 0.5, sd = 0.15),
  effect_grid = seq(0.2, 0.8, by = 0.1)
)

```

11.6 Example 6: Variance Uncertainty

Research question: How does uncertainty in the residual variance affect our sample size recommendation?

```

# Point estimate from pilot
result_fixed <- brms_inla_power(
  formula = y ~ treatment, effect_name = "treatment",
  effect_grid = 0.5, sample_sizes = c(30, 50, 80, 100),
  nsims = 500, error_sd = 1.0, seed = 42
)

# Integrating over variance uncertainty
result_uncertain <- brms_inla_power(
  formula = y ~ treatment, effect_name = "treatment",
  effect_grid = 0.5, sample_sizes = c(30, 50, 80, 100),
  nsims = 500,
  error_sd = list(dist = "halfnormal", sd = 0.25, location = 1.0),
  seed = 42
)

# Compare: integrating over variance uncertainty typically
# yields lower conditional power than conditioning on a point estimate

```

12. Reporting Results

When reporting Bayesian power or assurance results in a manuscript, the following template may be useful:

Reporting template: Sample size was determined using Bayesian assurance analysis (powerbrmsINLA version 1.2.0; Myers, 2026). We simulated [nsims] datasets at each combination of sample size ([list]) and true effect size ([list]), fitting a [model description] using INLA. The decision criterion was [describe: e.g., $P(\text{effect} > 0.2 \mid \text{data}) > 0.95$]. The analysis prior on the treatment coefficient was [describe]. To compute unconditional assurance, we specified a design prior of [describe: e.g., $\text{Normal}(0.5, 0.15)$] over the true effect, representing our genuine uncertainty. The recommended sample size of [N] per group provides [X]% assurance of meeting the decision criterion. Sensitivity analyses with [alternative priors] yielded assurance values of [range].

Key elements to report:

- Package name and version
- Number of simulations (nsims)
- Decision criterion and probability threshold
- Analysis priors used for model fitting
- Design prior used for assurance (if applicable)
- Distinguish conditional power from unconditional assurance
- Sensitivity analysis results
- Whether variance uncertainty was integrated

13. Computational Considerations

13.1 Computation Time

Factor	Impact
Number of simulations (nsims)	Linear scaling; 500 sims takes ~5x longer than 100
Sample size	Larger samples increase model fitting time moderately
Grid size	More effect/sample combinations = more total simulations
Model complexity	Random effects and non-Gaussian families increase time
Bayes factor computation	Marginal likelihood method is slower than Savage-Dickey

Typical Run Times

On a modern laptop (4 cores, 16GB RAM):

- Simple Gaussian model, 200 sims, 5 sample sizes, 5 effects: 2-5 minutes
- Mixed model with random intercepts, 500 sims, 7 sample sizes, 7 effects: 15-45 minutes
- Complex mixed model with Bayes factors, 500 sims, large grid: 1-3 hours

13.2 Recommendations for Efficiency

1. Start with few simulations (50-100) to verify code works correctly
2. Use a coarse grid initially to identify the region of interest
3. Consider the two-stage or sequential engines for large grids
4. Run final analysis overnight with full simulation count

Threading Configuration

In version 1.2.0, optimal INLA threading is auto-detected when `inla_num_threads = NULL` (the default):

```
# Auto-detect (default, usually optimal)
inla_num_threads = NULL

# Manual: "outer:inner" threads
inla_num_threads = "4:1"

# Conservative for memory-limited systems
inla_num_threads = "2:1"
```

13.3 How Many Simulations?

nsims	95% CI Width	Use Case
100	+/-10% around estimate	Initial exploration
500	+/-4-5% around estimate	Standard analysis
1000	+/-3% around estimate	Final reporting
2000+	+/-2% around estimate	High precision needed

For most applications, 500 simulations provides adequate precision.

14. Approximate Analogues to Traditional Power Analysis Tools

The following examples show how powerbrmsINLA can produce results approximately comparable to traditional tools. These are approximations, not exact equivalences, because the underlying inferential frameworks differ.

14.1 Approximate Analogues to G*Power Analyses

Two-Sample t-test

```
# G*Power: Two-tailed t-test, d = 0.5, alpha = 0.05
# Approximate powerbrmsINLA analogue:
ttest_power <- brms_inla_power(
  formula = y ~ group,
  effect_name = "group",
  effect_grid = 0.5,
  sample_sizes = seq(30, 100, by = 10),
  nsims = 500,
  prob_threshold = 0.975,
  seed = 123
)
```

With weak priors and approximately normal posteriors, `power_direction` gives an approximate analogue to traditional power. See Section 3.5 for important caveats.

Paired t-test

```
paired_power <- brms_inla_power(
  formula = y ~ condition + (1 | subject),
  effect_name = "condition",
  effect_grid = 0.5,
  sample_sizes = c(20, 30, 40, 50),
  nsims = 500,
  group_sd = 1,
  error_sd = 0.5,
  obs_per_group = 2,
  prob_threshold = 0.975,
  seed = 123
)
```

14.2 Approximate Analogues to JASP Bayesian Power

```
bf_design <- brms_inla_power(
  formula = y ~ group,
  effect_name = "group",
  effect_grid = 0.5,
  sample_sizes = seq(20, 100, by = 10),
  nsims = 500,
  compute_bayes_factor = TRUE,
  bf_method = "sd",
  bf_cutoff = 10,
  priors = c(
    prior(normal(0, 1), class = "Intercept"),
    prior(cauchy(0, 0.707), class = "b") # JASP default
  ),
  seed = 123
)
```

bf_hit_10 in summary is roughly comparable to JASP's BF "Power"

14.3 Key Differences

- **Simulation-based:** Results have Monte Carlo error; traditional tools use exact formulae for simple designs
- **Prior specification:** Bayesian methods require priors; results depend on prior choice
- **Model flexibility:** powerbrmsINLA handles complex mixed models that traditional tools cannot
- **Multiple criteria:** Traditional tools focus on significance; powerbrmsINLA supports multiple decision rules

14.4 When to Use Each Approach

Use G*Power/JASP When	Use powerbrmsINLA When
Simple designs (t-tests, basic ANOVA)	Complex mixed models, multilevel designs
Frequentist framework preferred	Bayesian analysis planned
Quick, exact results needed	Uncertainty in effect size is important
Grant applications requiring traditional power	Multiple decision criteria matter
Standard effect size measures (Cohen's d)	Non-Gaussian outcomes (counts, proportions)

15. Strengths and Limitations

15.1 Strengths

Computational Efficiency

INLA's deterministic approximations can be substantially faster than MCMC-based approaches for many latent Gaussian models. Analyses that would take hours with brms and Stan may complete in minutes with powerbrmsINLA. The speed advantage is model-dependent (Rue et al., 2009).

Model Flexibility

The package supports a broad range of generalised linear mixed models, including:

- Multiple response distributions (Gaussian, binomial, Poisson, negative binomial, beta, etc.)
- Random intercepts and slopes
- Multi-level structures
- Multiple fixed effects with interaction assessment

Decision-Theoretic Framework

Multiple decision criteria allow researchers to plan studies aligned with their actual inferential goals, whether that is meeting a prespecified decision criterion, establishing practical significance, or achieving precision.

Uncertainty Quantification

By treating effect sizes as uncertain (via design priors) and optionally integrating over variance uncertainty, assurance provides a more realistic assessment of study success probability than traditional power analysis.

15.2 Limitations

INLA Approximation

INLA provides approximations to posterior distributions. For most well-specified models with moderate data, these are excellent. However:

- Very weak priors or sparse data may reduce approximation quality
- Some complex covariance structures are not fully supported
- Extreme effect sizes may challenge the Laplace approximation
- Use `validate_inla_vs_brms()` to spot-check for your specific model

Random Effects Complexity

- Random intercepts: Fully supported
- Single random slope per term: Supported
- Multiple random slopes per term: May require manual configuration
- Correlated random effects: Limited automatic support

Bayes Factor Methods

- Savage-Dickey (`bf_method = "sd"`): Fast but requires proper Normal prior on the tested parameter
- Marginal likelihood (`bf_method = "marglik"`): More general but slower and can be sensitive to model specification

Prior Dependence

As with all Bayesian methods, results depend on prior specification. Sensitivity analyses using `plot_assurance_with_robustness()` are recommended.

Monte Carlo Error

Being simulation-based, results have inherent variability. More simulations reduce but cannot eliminate this uncertainty.

15.3 Best Practices

1. Start simple: Begin with small nsims and refine
2. Validate models: Ensure the formula works correctly before running full analyses
3. Check convergence: Examine results for any fitting issues (NA values, extreme estimates)
4. Conduct sensitivity analyses: Vary priors and effect assumptions
5. Report uncertainty: Include simulation error in power estimates
6. Document assumptions: Clearly state priors, effect expectations, and decision criteria
7. Spot-check INLA: Use `validate_inla_vs_brms()` for non-standard models
8. Distinguish conditional power from unconditional assurance in reporting

16. Quick Reference

16.1 Function Summary

Function	Purpose
<code>brms_inla_power()</code>	Main conditional power simulation engine
<code>brms_inla_power_sequential()</code>	Adaptive stopping for efficiency
<code>brms_inla_power_two_stage()</code>	Two-stage adaptive refinement
<code>brms_inla_power_parallel()</code>	Multi-core parallel computation
<code>brms_inla_power_design()</code>	Unified interface to all engines
<code>compute_assurance()</code>	Unconditional Bayesian assurance (New)
<code>assurance_prior_weights()</code>	Design prior weight construction (New)
<code>decide_sample_size()</code>	Automated sample size recommendation
<code>validate_inla_vs_brms()</code>	Spot-check INLA vs brms posteriors (New)
<code>validate_sd_spec()</code>	Validate variance specifications (New)
<code>beta_binom_power()</code>	Analytic beta-binomial assurance
<code>beta_weights_on_grid()</code>	Beta-distributed weights on an effect grid
<code>hdi_of_icdf()</code>	Highest density interval of inverse CDF

16.2 Plotting Functions

Function	Output
<code>plot_power_contour()</code>	Conditional power contour (effect x n)
<code>plot_power_heatmap()</code>	Conditional power heatmap
<code>plot_assurance_curve()</code>	Unconditional assurance vs n (New)
<code>plot_power_assurance_overlay()</code>	Conditional power + assurance overlay (New)
<code>plot_design_prior()</code>	Design prior density (New)
<code>plot_bf_assurance_curve()</code>	BF hit rate with CIs
<code>plot_bf_assurance_curve_smooth()</code>	Smoothed BF success probability curves
<code>plot_bf_expected_evidence()</code>	Mean log10 BF curves
<code>plot_bf_heatmap()</code>	BF evidence heatmap
<code>plot_decision_assurance_curve()</code>	Conditional power vs effect size
<code>plot_decision_threshold_contour()</code>	Power across decision thresholds
<code>plot_precision_assurance_curve()</code>	Precision probability curves
<code>plot_precision_fan_chart()</code>	Precision with bands
<code>plot_assurance_with_robustness()</code>	Multi-scenario comparison
<code>plot_interaction_surface()</code>	Two-effect surface/heatmap

Function	Output
<code>add_decision_overlay()</code>	Overlay sample size recommendations

16.3 Common Parameter Reference

Parameter	Typical Values	Notes
<code>nsims</code>	200-1000	500 balances precision and speed
<code>prob_threshold</code>	0.90, 0.95, 0.975	0.95 common; 0.975 ~ two-sided 0.05 (approx.)
<code>effect_threshold</code>	0, 0.1, 0.2	Minimum meaningful effect (on model coefficient scale)
<code>rope_bounds</code>	<code>c(-0.1, 0.1)</code> , <code>c(-0.2, 0.2)</code>	Often +/-0.1 or +/-0.2 SDs
<code>bf_cutoff</code>	3, 10, 30	10 = strong evidence (convention)
<code>error_sd</code>	0.5-2.0 or <code>list()</code>	1.0 gives standardised scale; <code>list()</code> for uncertainty
<code>group_sd</code>	0.3-1.0 or <code>list()</code>	Between-group SD; <code>list()</code> for uncertainty
<code>inla_num_threads</code>	NULL, "4:1", "2:1"	NULL = auto-detect (recommended)

17. Glossary

Assurance (unconditional): The probability of achieving the desired inferential outcome, averaged over a design prior on the true effect. Formally introduced by O'Hagan & Stevens (2001). Distinguished from conditional power, which assumes a fixed effect size.

Bayes Factor (BF): A ratio quantifying the relative evidence for one hypothesis versus another. $BF_{10} > 1$ favours the alternative; $BF_{10} < 1$ favours the null. Sensitive to the prior on the tested parameter.

Conditional Power: The probability of meeting a prespecified decision criterion given a fixed true effect size. The output of `brms_inla_power()` at each effect-grid value.

Credible Interval (Crl): A Bayesian interval estimate; there is a stated probability (e.g., 95%) that the true parameter lies within the interval, given the data and prior. Not the same as a frequentist confidence interval.

Design Prior: A probability distribution representing uncertainty about the true effect size, used to compute unconditional assurance by weighting conditional power across effect values. Should be justified separately from the analysis prior.

INLA: Integrated Nested Laplace Approximation; a computational method for approximate Bayesian inference that can be substantially faster than MCMC for many latent Gaussian models (Rue et al., 2009).

Marginal Likelihood: The probability of the data under a model, integrated over all parameter values. Used to compute Bayes factors when `bf_method = 'marglik'`.

MCMC: Markov Chain Monte Carlo; a family of algorithms for sampling from probability distributions, commonly used in Bayesian analysis but computationally intensive.

Posterior Distribution: The probability distribution of a parameter after observing data; combines prior information with the likelihood of the data.

Prior Distribution (Analysis Prior): A probability distribution representing beliefs about a parameter, used when fitting the model to each simulated dataset.

ROPE (Region of Practical Equivalence): A range of effect sizes considered practically equivalent to zero or negligible. ROPE-based decisions can test for either practical significance (posterior outside ROPE) or practical equivalence (posterior inside ROPE).

Savage-Dickey Ratio: A method for computing Bayes factors by comparing the prior and posterior densities at a point null hypothesis (typically zero). Requires a proper prior on the tested parameter.

Variance Uncertainty: The recognition that nuisance parameters such as the residual standard deviation are themselves uncertain. `powerbrmsINLA 1.2.0` allows this uncertainty to be integrated into power calculations via distributional specifications for `error_sd` and `group_sd`.

18. References

- Burkner, P.-C. (2017). brms: An R Package for Bayesian Multilevel Models Using Stan. *Journal of Statistical Software*, 80(1), 1-28. <https://doi.org/10.18637/jss.v080.i01>
- Kruschke, J. K. (2014). *Doing Bayesian Data Analysis: A Tutorial with R, JAGS, and Stan* (2nd ed.). Academic Press. ISBN: 9780124058880.
- Kruschke, J. K. (2018). Rejecting or Accepting Parameter Values in Bayesian Estimation. *Advances in Methods and Practices in Psychological Science*, 1(2), 270-280. <https://doi.org/10.1177/2515245918771304>
- O'Hagan, A., & Stevens, J. W. (2001). Bayesian Assessment of Sample Size for Clinical Trials of Cost-Effectiveness. *Medical Decision Making*, 21(3), 219-230. <https://doi.org/10.1177/0272989X0102100307>
- O'Hagan, A., Stevens, J. W., & Campbell, M. J. (2005). Assurance in clinical trial design. *Pharmaceutical Statistics*, 4(3), 187-201. <https://doi.org/10.1002/pst.175>
- Rue, H., Martino, S., & Chopin, N. (2009). Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations. *Journal of the Royal Statistical Society: Series B*, 71(2), 319-392. <https://doi.org/10.1111/j.1467-9868.2008.00700.x>

End of Guide

For updates and further documentation, see:

<https://cran.r-project.org/package=powerbrmsINLA>

Report issues and contribute at the package GitHub repository:

<https://github.com/Tony-Myers/powerbrmsINLA>